

Es. 1. CODIFICA CARATTERI

In Danese la parola di quattro lettere LÆSE significa leggere. Qui di seguito sono riportate tre codifiche di tale parola: una di queste è ASCII esteso Latin 1, un'altra è UTF-8, la rimanente è UCS-2 (codifica su 2 byte del codepoint unicode del Basic Multilingual Plane). Le tre codifiche sono riportate per brevità in esadecimale (ogni coppia di cifre esadecimali corrisponde ad un byte). Identificare ciascuna delle tre codifiche e spiegare come si possono riconoscere (Nota: Le lettere codificate sono tutte maiuscole.).

CODIFICA a) 4C C3 86 53 45

CODIFICA b) 00 4C 00 C6 00 53 00 45

CODIFICA c) 4C C6 53 45

Nota: lo schema di codifica di UTF-8 è riportato di seguito.

Codepoint unicode fino a	Byte 1	Byte 2	Byte 3	Lunghezza codifica
7 bit	0bbbbbbb	Non usato	Non usato	1 byte
11 bit	110bbbbbb	10bbbbbbb	Non usato	2 byte
16 bit	1110bbbb	10bbbbbbb	10bbbbbbb	3 byte

VEDIAMO LA CODIFICA B) 00 4C 00 C6 00 53 00 45 SONO 8 BYTE E VISTO CHE LA PAROLA CODIFICATA HA 4 LETTERE LÆSE CHE OGNI GRUPPO DI 2 BYTE RAPPRESENTA UNA LETTERA: ^L004C ^Æ00C6 ^S0053 ^E0045, QUINDI SONO IN CODIFICATE IN UCS-2 (2 BYTE A LUNGHEZZA FISSA), INFATTI L, S, E CORRISPONDONO ALL'ASCII BASE, MENTRE Æ ALL'ASCII ESTESO, CON ENTRAMBI UN PADDING DI ZERI.

LA CODIFICA C) 4C C6 53 45, SONO 4 BYTE E SE OGNI LETTERA ASSOCIAMO UN BYTE, NOTIAMO CHE COME PRIMA 4C(L), 53(S) E 45(E) SONO SCRITTI IN ASCII BASE MENTRE C6(Æ) IN ASCII ESTESO LATIN-1. QUINDI QUESTA È UNA CODIFICA IN ASCII ESTESO USANDO APPUNTO 1 BYTE PER OGNI LETTERA.

LA CODIFICA A) 4C C3 86 53 45, NOTIAMO CHE SONO 5 BYTE PER 4 LETTERE, QUINDI UNA LETTERA È CODIFICATA SECONDO UN'ALTRA LUNGHEZZA RISPETTO ALLE ALTRE, VEDENDO CHE ANCORA 4C, 53 E 45 SONO UGUALI ALL'ASCII ESTESO, C3 86 È USATO PER CODIFICARE UN'UNICA LETTERA. ESSENDO A LUNGHEZZA VARIABILE DA 1 A PIÙ BYTE, È SICURAMENTE CODIFICATO IN UTF-8, MA CONTROLLIAMO:

$C3_{16} \rightarrow \overset{6}{1100} \overset{3}{0011}$ $86 \rightarrow \overset{8}{1000} \overset{6}{0110}$ QUINDI:

BYTE 1	BYTE 2
<u>1100</u> 0011	<u>1000</u> 0110

Es.2 – Codifica del testo

- (a) Che differenza c'è tra la codifica dei caratteri in ASCII esteso, UTF-8 e Unicode su 16 bit (UCS-2)?
 (b) La codifica ASCII esteso Latin-1 della parola Città (in esadecimale) è: 43 69 74 74 E0

ricavare la codifica UTF-8 e Unicode su 16 bit Bit con ordinamento Big Endian della stessa parola

Tabella parziale dello schema di codifica UTF-8

Fino a 7 bit	0xxxxxxx			
Da 8 a 11 bit	110xxxxx	10xxxxxx		
Da 12 a 16 bit	1110xxxx	10xxxxxx	10xxxxxx	

A) L'ASCII ESTESO È UNA CODIFICA A LUNGHEZZA FISSA SU UN BYTE (8 BIT) E SI POSSONO RAPPRESENTARE SOLO UN TIPO DI ASCII ESTESO PER TESTO, L'UTF-8 È UNA CODIFICA DI UNICODE A LUNGHEZZA VARIABILE DA 1 A 6 BYTE (AD OGGI DA 1 A 4 BYTE) PERMETTE DI RISPARMIARE SPAZIO E COMPATIBILE CON L'ASCII, L'UCS-2 INVECE È LA PRIMA CODIFICA DI UNICODE (UNICODE 1.0 SU 16 BIT) A LUNGHEZZA FISSA SU 2 BYTE (65536 CARATTERI).

B) 43 69 74 74 E0

IN UTF-8

0100 0011 | 0110 1001 | 0111 0100 | 0111 0100 | 1110 0000

C → 0100 0011 | I → 0110 1001 | T → 0111 0100 | T → " "

À → 1110 0000 → 8 BIT → 1100 0011 | 1010 0000
 C 3 A 0

CITTÀ → 0x43 0x69 0x74 0x74 0xC3A0

IN UCS-2:

B.E. : C → 00 43 | I → 00 69 | T → 00 74 | T → 00 74 | À → 00 E0

L.E. : C → 43 00 | I → 69 00 | T → 74 00 | T → 74 00 | À → E0 00

Domanda 1 – Codifica del testo

- a) Quanti bit occupa un carattere nelle tre codifiche "ASCII base", "ASCII esteso" e "UCS-2"?
 b) Esistono delle codifiche di caratteri a lunghezza variabile? Se sì indicare il nome di almeno una codifica di questo tipo e discutere i vantaggi di questo tipo di codifica.
 c) Fare un esempio di codifica UCS-2 mostrando la rappresentazione in memoria della parola giapponese 東京 (Tokio) sia nell'ordinamento Big Endian che in quello Little Endian. I codici Unicode dei due ideogrammi sono: 0x6771 (東) 0x4EAC (京). Nota 1: il prefisso 0x indica che ciò che segue è un numero esadecimale; ogni cifra esadecimale corrisponde a 4 bit, quindi 2 cifre corrispondono a un byte. Nota 2: per la rappresentazione in memoria considerate per ciascun carattere due byte memorizzati in indirizzi consecutivi I e I+1 e indicate il contenuto (in esadecimale) del byte di indirizzo I e quello del byte di indirizzo I+1 in ciascuno dei due ordinamenti, Big Endian e Little Endian.

- A) UN CARATTERE IN ASCII BASE OCCUPA 7 BIT, IN ASCII ESTESO OCCUPA 8 BIT (1 BYTE) MENTRE IN UCS-2 NE OCCUPA 16 BIT (2 BYTE).
- B) SI ESISTONO CODIFICHE DI CARATTERI A LUNGHEZZA VARIABILE, UN ESEMPIO È L'UTF-8 CHE UTILIZZA DA 1 FINO A 6 BYTE (OGGI DA 1 A 4 BYTE), E I SUOI VANTAGGI SONO:
- RISPARMIARE SPAZIO RISPETTO A CODIFICHE A LUNGHEZZA FISSA
 - IL PRIMO BYTE È DIVERSO DAI SUCCESSIVI (NON SI PERDE L'ALLINEAMENTO)
 - È COMPATIBILE CON L'ASCII BASE (SONO SCRITTI ALLO STESSO MODO)
 - NON È LEGATO ALL'ORDINE DEI BYTE.
- C) 0X6771 0X4EAC

IN UCS-2:

PER RAPPRESENTARE OGNI CARATTERE VENGONO USATI 2 BYTE QUINDI SONO SCRITTI ALLO STESSO MODO:

	L	L+1		L	L+1
B.E.:	67	71	L.E.:	71	67
	4E	AC		AC	4E

Domanda 1 – Codifica del testo e ordinamento byte in memoria

- Per quale ragione se si usa la codifica ASCII esteso non è possibile utilizzare in uno stesso testo caratteri da lingue diverse, per esempio il cinese e il cirillico, mentre se si usa una codifica basata su UNICODE (come UCS-2 oppure UTF-8) ciò è possibile?
- Le rappresentazioni dei codepoint di UNICODE su più byte (come UCS-2 che usa 2 byte o UTF-32 che ne usa 4) possono essere memorizzate secondo l'ordinamento Big Endian oppure Little Endian: spiegare qual è la differenza. Per esempio se dovessimo rappresentare in 4 byte consecutivi (per esempio in 4 byte di indirizzo 10, 11, 12 e 13) i bit del codepoint 0001F60D (8 cifre esadecimali sono 4 byte, 2 cifre per byte) che corrisponde ad un emoticon (faccina che ride con occhi a cuore), come sarebbero le due rappresentazioni in memoria? Indicare il contenuto di ciascun byte nei due casi.

- A) QUANDO SI USA L'ASCII ESTESO, OUVERO ANCHE DETTO CODEPAGE, NON PUOI SCRIVERE CARATTERI DI DIVERSI CODEPAGE IN UN UNICO TESTO, UN CALCOLATORE UTILIZZA UN CODEPAGE ALLA VOLTA E VEDREBBE CARATTERI SBAGLIATI, INOLTRE SE DUE CALCOLATORI USANO DUE CODEPAGE DIVERSI MAGARI UNO IN LATIN-1 E L'ALTRO IN GIAPPONESE SAREBBE IMPOSSIBILE COMUNICARE.

PER QUESTO VENNE INTRODOTTO L'UNICODE CHE ASSOCIA PER OGNI CARATTERE DI QUALUNQUE LINGUA E SIMBOLO DI QUALUNQUE AMBITO UN CODEPOINT,

OVVERO UN CODICE UNIVOCO COME IL CODICE FISCALE.

B) IL BIG ENDIAN E IL LITTLE ENDIAN SONO DUE MODI PER RAPPRESENTARE UN DATO, AFFERMANDO CHE QUEST'ULTIMO SIA IN CELLE DI MEMORIA DA 1 BYTE :

- BIG ENDIAN : IL BYTE PIÙ SIGNIFICATIVO DEL DATO VA NELLA CELLA DI MEMORIA PIÙ BASSA.

- LITTLE ENDIAN : IL BYTE MENO SIGNIFICATIVO DEL DATO VA NELLA CELLA DI MEMORIA PIÙ BASSA.

IL PROBLEMA SORGE QUANDO UN TESTO MAGARI È STATO CREATO IN LITTLE ENDIAN MA IL DESTINATARIO NON LO SA E MAGARI LO LEGGE IN BIG ENDIAN (TIPICO CASO DI SCAMBIO DI DATI TRA ARCHITETTURA AMD E INTEL), PROPRIO PER QUESTO ALL'INIZIO DEL FILE C'È IL BOM (BINARY ORDER MARKER) OVVERO UN PREFISSO DA 16 BIT CHE SE IMPOSTATO A FFFE VUOL DIRE CHE IL FILE È IN BIG ENDIAN, MENTRE SE IMPOSTATO A FEFF VUOL DIRE CHE IL FILE È IN LITTLE ENDIAN.

QUINDI L'ESEMPIO CON : 0001FG0A DOVE OGNI CIFRA SONO 2 BYTE QUINDI 4 CIFRE IN TUTTO IN 4 CELLE DI MEMORIA (10, 11, 12, 13) :

B.E. :

10	11	12	13
00	01	F6	0A

L.E. :

10	11	12	13
0A	F6	01	00